

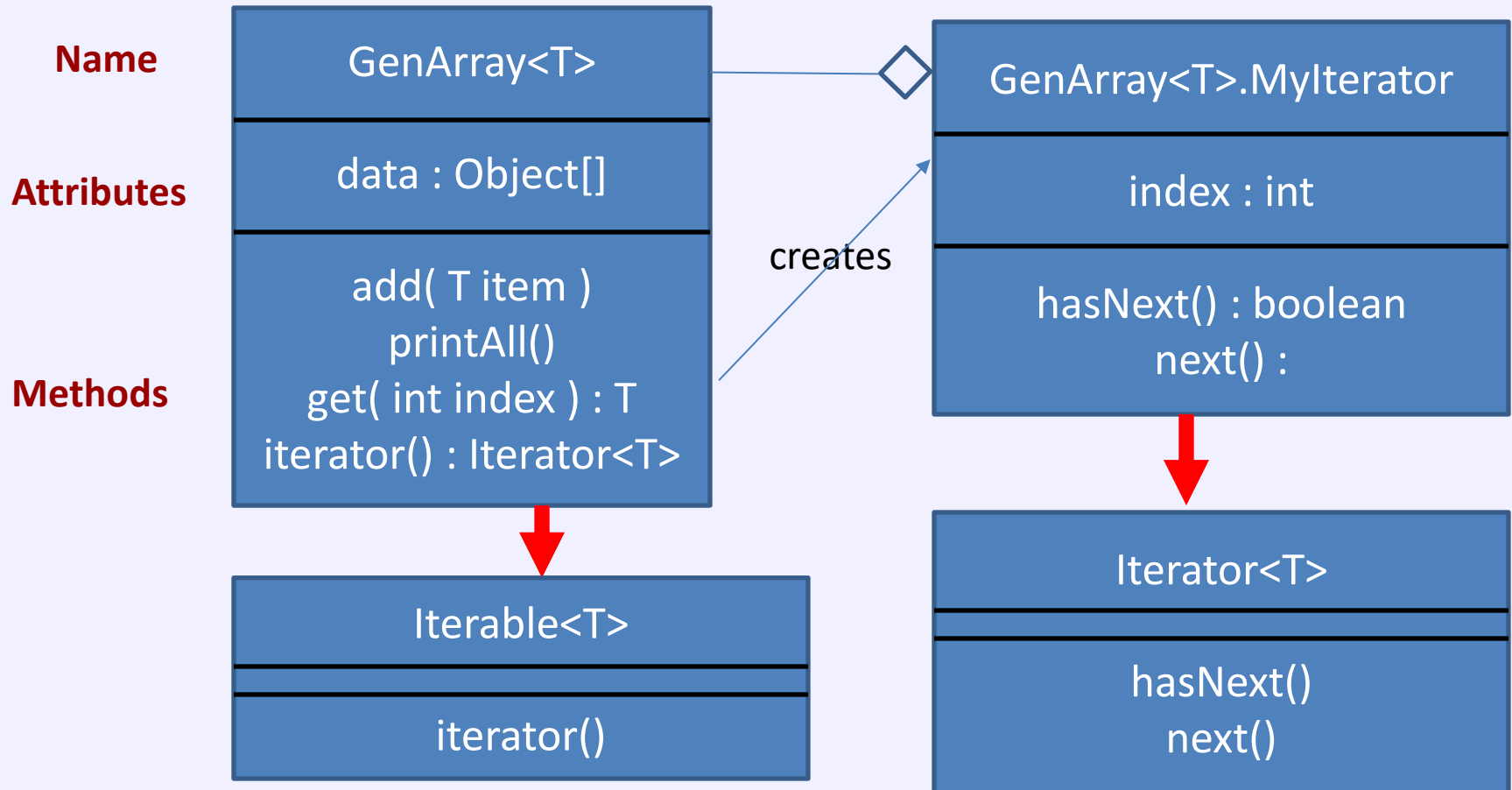
COMP1009

Programming Paradigms

Lecture 00-13

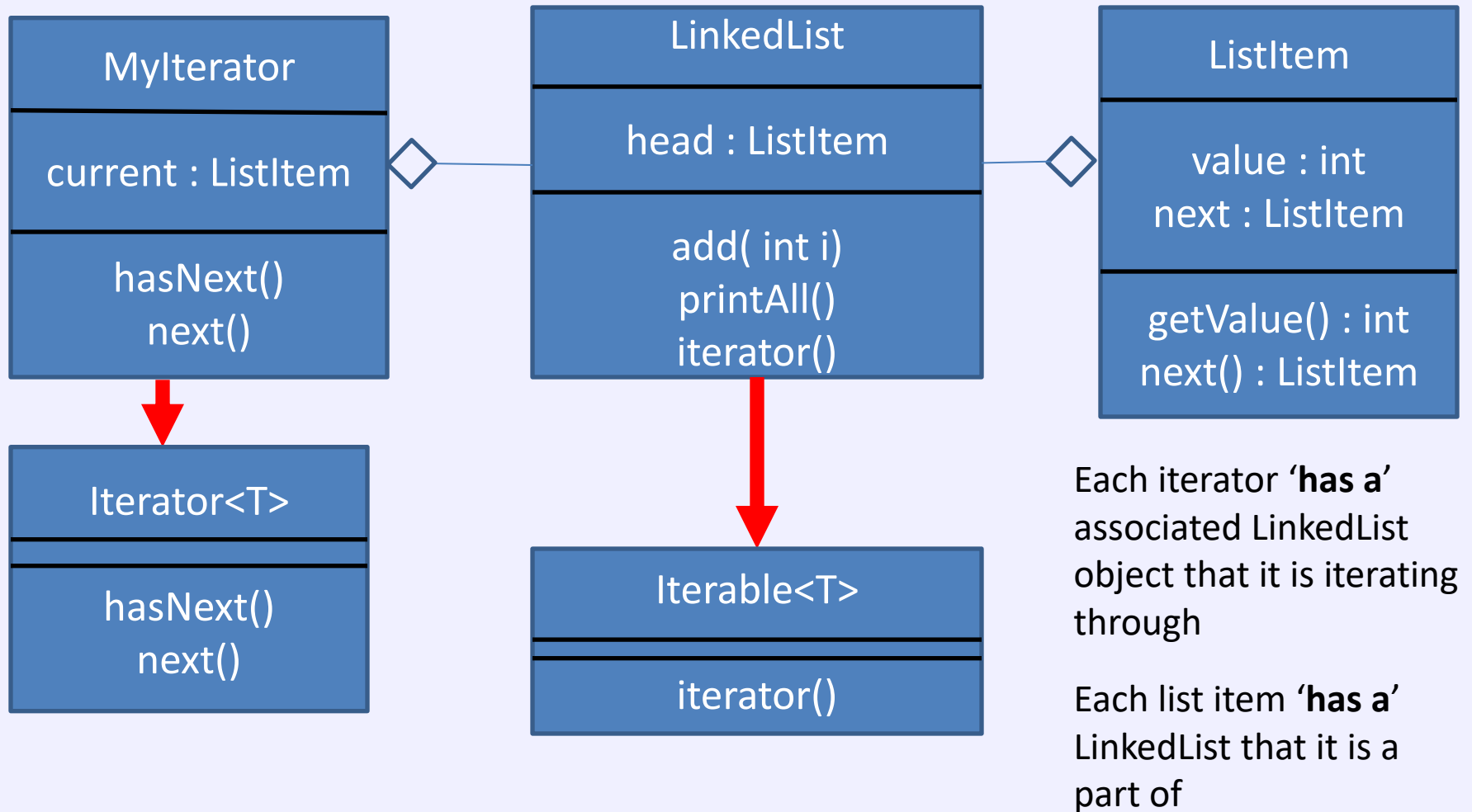
Anonymous classes

Last Lecture: Class Diagram for GenArray



- Note: I massively simplified the class diagrams, to make them easier to understand and be closer to Java format
- In an exam we check for the main concepts – simplify!

Last Lecture: Class diagram for Linked List

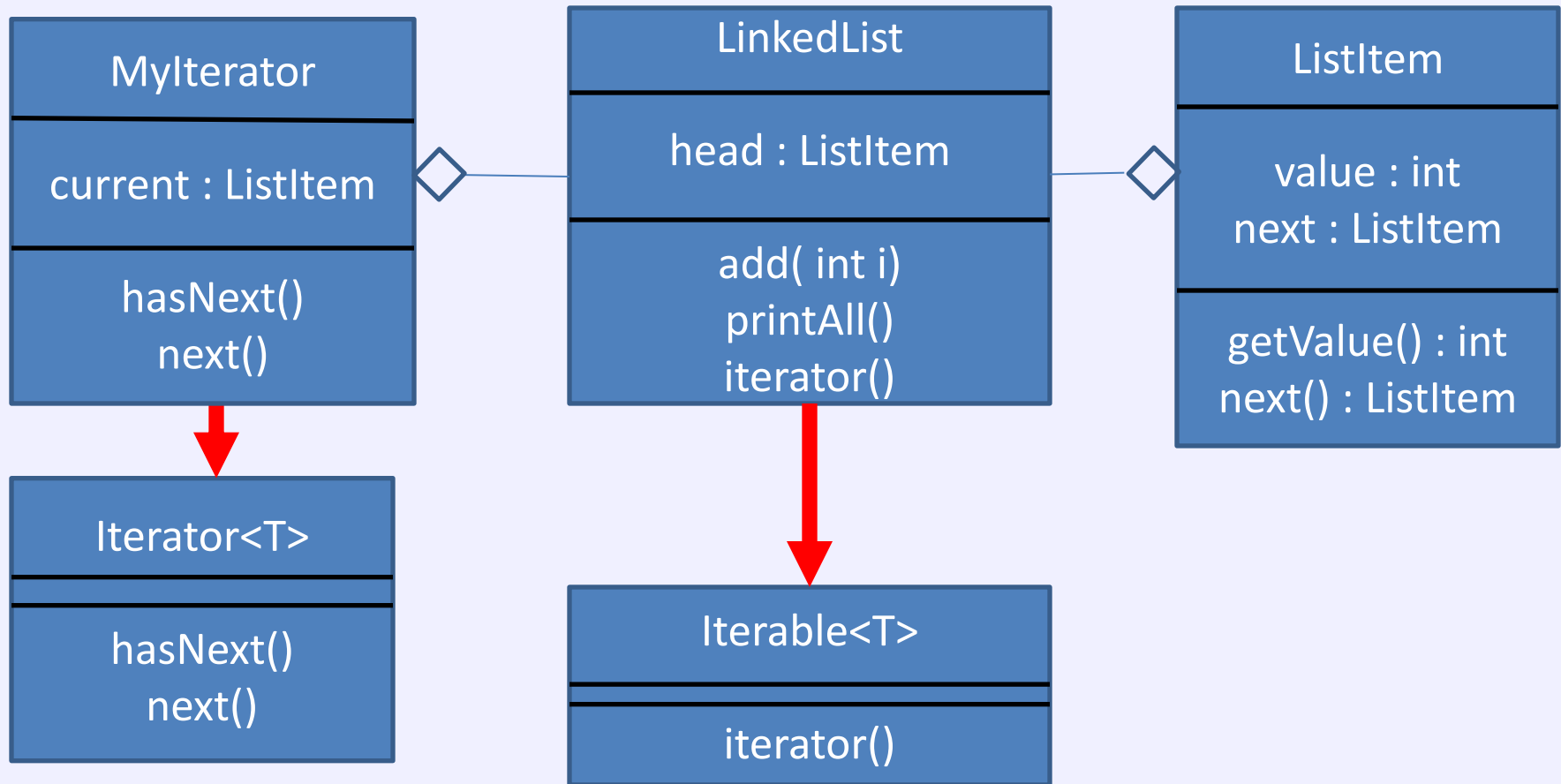


- We created another inner class for MyIterator

Alternatives to Inner Classes

Anonymous classes

Simplified Class Diagram



- MyIterator class name was only used in one place – when object is created
- Everywhere else we treated it as the Iterator interface
- All that mattered was its implementation of the methods `next()` and `hasNext()`
- Why do we need to give it a name at all? And why a separate class?

Alternative 1: Anonymous class

- Key point: we only used **the name** of the class where we created the object, nowhere else!
 - We never used its name anywhere else
 - This is really **important** – because using this approach we won't have a class name any more to refer to it elsewhere
- It may be annoying to keep creating a whole new class each time we need **one object** somewhere in the code
 - Also, the place where I use the class (to create the object) is not the place where I define the class, which is annoying
 - Especially for event handlers (e.g. for buttons) it would be useful to just say 'this button does this...'
- So why bother creating a new class for this in advance?
 - And why do I have to create it somewhere else in the code
- We could create an anonymous class for this...

Example with GenArray iterator...

```
class MyIterator implements Iterator<T>
{
    int index = 0;

    @Override
    public boolean hasNext()
    {
        return index < data.length;
    }

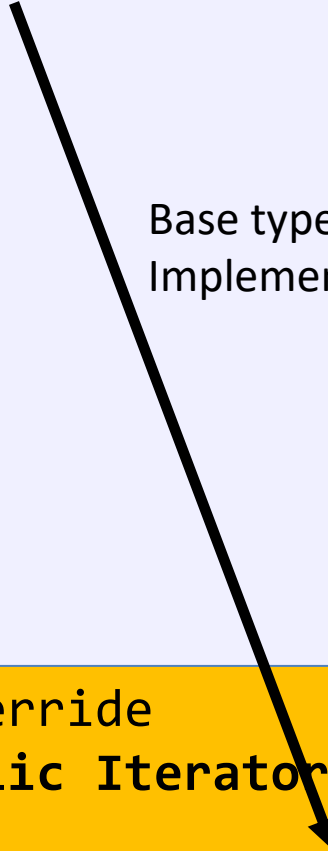
    @Override
    public T next()
    {
        return get( index++ );
    }
}
```

```
@Override
public Iterator<T> iterator()
{
    return new MyIterator();
}
```

Move the code to the object creation point

```
class MyIterator implements Iterator<T>  
{  
    int index = 0;  
  
    @Override  
    public boolean hasNext()  
    {  
        return index < data.length;  
    }  
  
    @Override  
    public T next()  
    {  
        return get( index++ );  
    }  
}
```

Base type replaces class name
Implementation appears after ()



```
@Override  
public Iterator<T> iterator()  
{  
    return new MyIterator();  
}
```

The resulting iterator() method

```
public Iterator<T> iterator()  
{  
    return new Iterator<T>() ←  
    {  
        int index = 0;  
  
        @Override  
        public boolean hasNext()  
        {  
            return index < data.length;  
        }  
  
        @Override  
        public T next()  
        {  
            return get( index++ );  
        }  
    };  
}
```

new Iterator<T>() { ... }
Create an anonymous (nameless) sub-class of Iterator<T>, with the following data and methods

Use this ONLY when you only use the class in one place – you will have no name to refer to it

Looking at it another way: if there is { after the parameters to pass to constructor in new, then it means to create a sub-class object with these overrides

Anonymous Classes

- Key elements:

```
return new Iterator<T>()  
{ ... include details of sub-class ... }
```

- rather than separating class definition and creation:

```
class MyIterator implements Iterator<T>  
{ ... include details of class ... }
```

```
return new MyIterator();
```

- The class is defined without a name
- The class is defined at the place the object is created
- When you see the { after the (), read it as “Create a new sub-class of this class, with this implementation”

Code Examples

- Example with the code
 - Copy paste
 - And with the list
- Autofix works with anonymous classes too
 - E.g. Create new one:
`return new Iterator<T>() {};`
 - And let the auto-fix generate the blank methods
- Anonymous classes really are inner classes with no name!

Use with buttons

Anonymous classes

Buttons Example

- In a buttons example, our main class implemented ActionListener

```
public class ButtonsExample6 implements ActionListener  
{
```

- Button adds the action listener

```
button1.addActionListener(this);
```

- And main class has an actionPerformed implementation

```
@Override
```

```
public void actionPerformed(ActionEvent e)  
{
```

```
    label.setText(label.getText()+".");;
```

```
    button1.setBackground( new Color( rand.nextInt(256),  
                                rand.nextInt(256), rand.nextInt(256) ));
```

```
    guiFrame.pack();
```

```
}
```

- Note: could have used an inner class instead

Alternative using anonymous class

```
button1.addActionListener(  
    new ActionListener() {  
        public void actionPerformed(ActionEvent e)  
        {  
            label.setText(label.getText()+".");  
            button1.setBackground(  
                new Color( rand.nextInt(256),  
                           rand.nextInt(256),  
                           rand.nextInt(256) ));  
            guiFrame.pack();  
        } } );
```

Demo/Example

- Demo with a button – converting to anonymous class
- A second button, e.g.

```
    JButton button2 = new JButton("Clear");  
    guiFrame.getContentPane().add(button2);  
    button2.addActionListener( new ActionListener() {  
        @Override  
        public void actionPerformed(ActionEvent e)  
        {  
            label.setText("");  
            guiFrame.pack();  
        }  
    } );
```

Summary

- An anonymous class is **an inner class** without a name
- Instead of giving a class name and saying that the class extends/implements some class/interface, name the **super-class** (base class/interface) and then give the implementation of the **changes/additions**

```
new ActionListener() // Superclass
{ // These {} matter, they mean sub-class it
    public void actionPerformed(ActionEvent e)
    { ... }
}
```

- This really is an inner class, so is associated with an outer class object and has access to its variables
- In the generated code, these are often named \$1, \$2 etc

An even simpler alternative to Inner Classes for single- function interfaces

Lambdas

(beware: limitations to where
they can be used)

Lambdas in Java

- Lambdas in Java are basically simpler syntax for an anonymous class when it subclasses:
an **interface** which has **only one method**
- Both of these **two** criteria are important
- The compiler has to know to expect an interface, and to be able to work out the type of interface (from context)
 - E.g. type of parameter expected by a method
- You give it an implementation of **the** method
 - There is only one method on the interface so **it knows** the method name
 - **It knows** the return type
 - And **it knows** the parameter types
 - It **doesn't know** the parameter names
 - It **doesn't know** what the method does

Example

```
button2.addActionListener(  
    e -> label.setText("") );
```

- Compiler knows addActionListener() expects an ActionListener object
- Compiler knows ActionListener has a single method called actionPerformed()
- Compiler knows actionPerformed() has a single parameter – we tell it the parameter name(s), it already knows the type(s)
- We tell it what actionPerformed actually does

Demo/Example – with buttons

- Demo with a button
 - Using Buttons Example from lecture 9

```
button1.addActionListener( new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e)  
    {  
        label.setText(label.getText()+".");;  
        button1.setBackground(  
            new Color(    rand.nextInt(256),  
                          rand.nextInt(256),  
                          rand.nextInt(256) ));  
        guiFrame.pack();  
    }  
});
```

Using a Lambda for our buttons

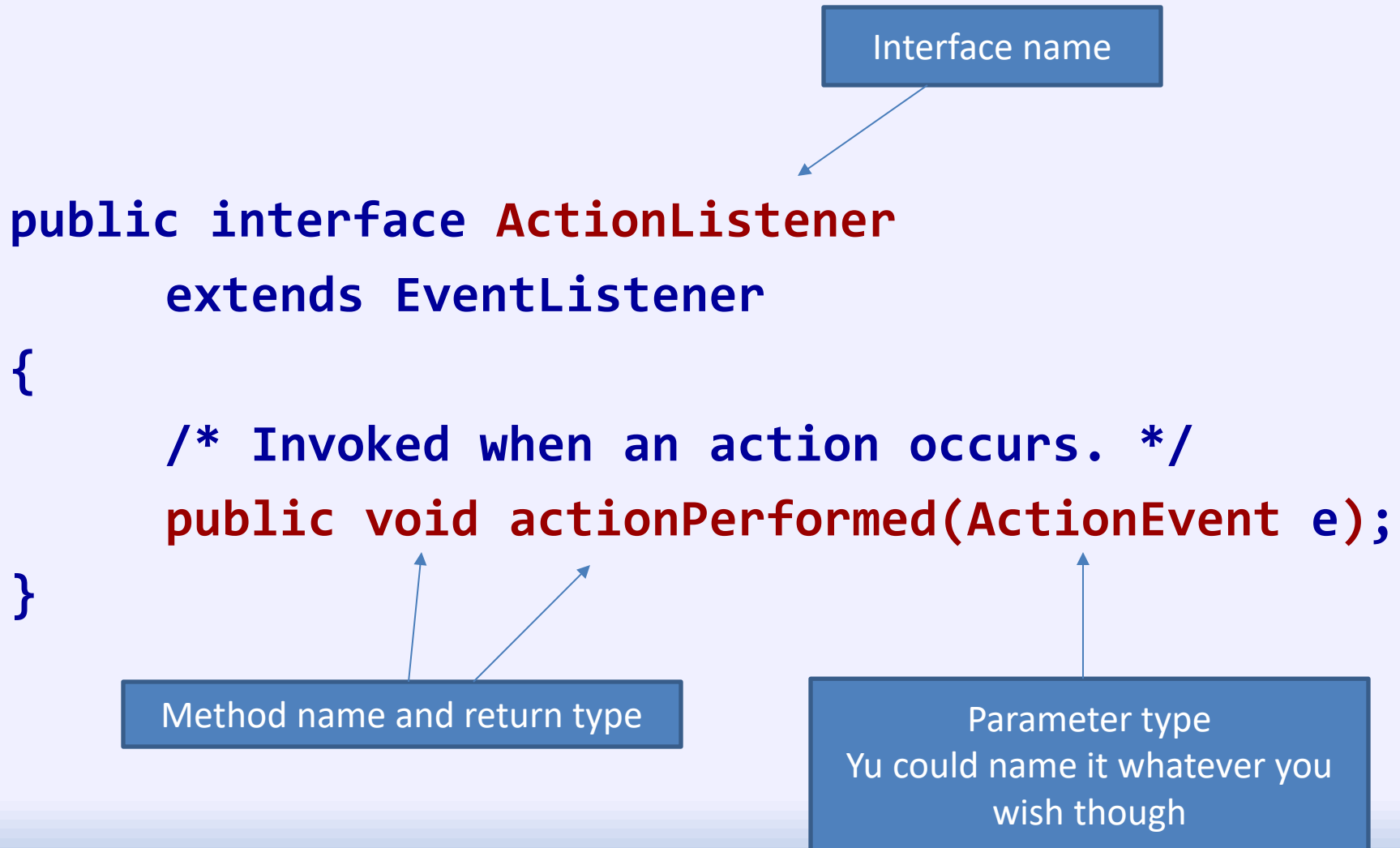
```
button1.addActionListener( new ActionListener()
_____ {
_____ @Override
_____ public void actionPerformed(ActionEvent e)
_____ {
_____     label.setText( label.getText()+".");
_____     frame.pack();
_____ }
_____ } );
```

Keep this

Keep the implementation too

```
button1.addActionListener( e ->
{
    label.setText( label.getText()+".");
    frame.pack();
} );
```

Reminder: ActionListener interface



Lambda expression definitions

- Lambdas use the `->` operator
 - If you see `->` used in Java think Lambda! (Not like C `->`)
 - Note: C++ uses `[]` instead and has more **flexibility** & **complexity**!
- To the left of the `->` are the parameters (possibly in brackets)
- To the right is the expression to evaluate to return the result value (don't need `return` or `{}` if you have one expression)
- For example:

Parameters

Method implementation

(x,y)->x+y

 - Expects two parameters, called x and y and returns sum of x and y
- It will:
 - create a class which does this (implementing the interface)
 - create an instance of the class (an object)
 - provide a reference to the object
- Note: Compiler/JVM's Lambda implementation may differ from anonymous class implementation, but will do same thing

Variable capture

- Lambda expressions can capture **final** variables from the surroundings
 - Remember: final means constant/unchangeable/immutable
 - Or **effectively final** variables – any variables you COULD have made final
 - i.e. Any variables that cannot possibly change AFTER the Lambda has been created
- Think of it as putting a **copy** of this variable inside the Lambda
- A Lambda can use member variables
 - A Lambda is effectively an inner class (but may have simplified implementation)
 - A Lambda can access the 'this' object reference to the current (outer) object
 - 'this' will never change for the Lambda so it's OK to capture it
 - In this case it captured 'this' (hidden from you)
 - It used **(this.)label** and **(this.)frame**
- We can make any variable final as long as we do not need to change it after it is assigned to
 - Lambdas can capture any variable that could have been final

Example of variable capture

```
public class TestLambdaCapture
{
    interface Modify
    {
        public int val();
    }

    public static void main( String[] args )
    {
        Modify[] arr = new Modify[10]; // Array of modify objects

        for ( int i = 0 ; i < 10 ; i++ )
        {
            final int j = i;
            arr[i] = () -> j*2; // Implements val() to return j * 2
        }

        for ( Modify m : arr )
            System.out.println( "" + m.val() );
    }
}
```

Lambda expressions

- You can think of OO Lambda Expressions as a way of wrapping up a function in an object
 - An implementation of any interface (i.e. an object) is really just a way of wrapping up the implementations of various functions and some data
- Once we have an object we keep an object reference to it
- We can use sub-type polymorphism and treat it as the base-class (or interface) in the code
 - Inheritance means IS-A
- So, we call methods without worrying about what they will do for the real object
- You can access member variables, or 'final' local variables, which it will 'capture'

Summary of alternatives

- If you need an object to handle a function...
 - E.g. an event handler?
- ... you could:
 - Create an object of a new top-level class
 - Create an object of an inner class
 - Has access to members of outer class object
 - Create an object of an anonymous class
 - If you only need it in one place – as an inner class!
 - Create a Lambda
 - If your interface has only one method

Future Lectures

- Only the two labs before end of term – use them
 - This week and next week
- Next lecture we have the last OO/Java lecture
- Then we have the Joint Programming Paradigms wrap-up
- We then have one “OO/Java exam revision/prep lecture”
- There will also be some optional material to investigate some things further – only if you are interested (not exam)
 - ‘functional programming’-type lists in Java (FP in Java)
 - Concurrency/Threads and the issues with them